Weichi Zhao

Distributed LLM Inference & Scheduling Engineer

Heterogeneous Runtime · KV-Aware Routing · P/D Autoscaling

Hangzhou, China   wzhao4z@outlook.com   github.com/WZStephen   linkedin.com/in/wzhao42   wzstephen.github.io

PROFESSIONAL SUMMARY

Distributed LLM inference and scheduling engineer with five years of cloud-native AI/HPC experience and nearly three years in production serving. Leads Kubernetes-native runtime/control-plane work for heterogeneous non-NVIDIA accelerators across worker lifecycle, Mooncake/RoCE KV transfer, KV-aware routing, P/D capacity planning, and autoscaling; also delivered multi-tenant batch inference and model gateway systems. Primary languages: Go and Python, with Rust routing experience.

EXPERIENCE

Zhejiang Lab · High-Performance Computing Facility Research Center — 2024 - Present

Senior Research Specialist | LLM Serving / Model-as-a-Service (MaaS) Platform

- Led a distributed inference runtime/control plane for heterogeneous non-NVIDIA accelerators spanning worker bootstrap, model loading, the KV event plane, Mooncake/RoCE KV transfer, KV-aware routing, adaptive P/D capacity planning, and Kubernetes lifecycle orchestration; modeled process, model, event, data-plane, and traffic readiness as distinct states.
- Designed and productionized an auditable adaptive P/D Planner combining request rate, ISL/OSL, prefill cached/compute tokens, KV hit rate, queue/running state, and TTFT/TPOT; selected capacity profiles by model, accelerator, quantization, and parallelism to produce separate prefill/decode targets and scale intents.
- Designed and implemented an OpenAI Batch API-compatible execution platform covering file and batch lifecycles, task sharding, worker execution, Kubernetes Lease-based leader election, tenant concurrency, recovery, and result/error writeback; continuously supports production data-distillation and batch-generation workloads.
- Integrated SGLang HTTP/gRPC P/D routing, multi-tenant admission, and end-to-end tracing across heterogeneous non-NVIDIA backends; traced production failures across request ID → Gateway → Router → Worker with KV-transfer, queue, and worker-state telemetry.

Zhejiang Lab · Intelligent Supercomputing Research Center — 2021 - 2023

Engineering Specialist | Cloud-Native AI / HPC Platform

- Customized Kubernetes device-plugin and resource-management paths for heterogeneous accelerator discovery, node resource reporting, container-runtime validation, device-level allocation, and fault isolation.
- Adapted PyTorch/models and distributed job environments to Linux distributions and heterogeneous accelerator stacks, establishing the platform foundation for production serving and scheduling across non-NVIDIA accelerators.

SELECTED SYSTEMS

Heterogeneous Runtime & KV-Aware Routing

Worker Bootstrap · KV Event · Mooncake / RoCE

- Extended worker startup from process launch into a runtime initialization contract: attached engine lifecycle to DistributedRuntime; registered request-plane endpoints and Model Deployment Cards with accelerator type, WorkerType, P/D role, dependency DNF, KV capacity, and RoCE topology metadata.
- Initialized a KvEventPublisher per DP rank and normalized native SGLang/vLLM `BlockStored`, `BlockRemoved`, and `AllBlocksCleared` events into the event plane; transferred the actual KV tensors between prefill and decode through Mooncake Transfer Engine over RoCE.
- Combined prefix KV overlap, worker load, accelerator compatibility, and network topology for routing; fell back to a token-level approximate index when events were unavailable, while controllers coordinated model, event-plane, Mooncake/RoCE channel, and Router readiness.

SELECTED SYSTEMS (CONTINUED)

Adaptive P/D Planner and Scaling Guardrails
Capacity Profile · ScaleIntent · Guardrails

- Derived request rate, input/output tokens, prefill compute/cached tokens, and KV hit rate from rolling windows; selected profiled prefill/decode capacity curves by model, accelerator, quantization, TP/DP, and engine version with interpolation and conservative fallback.
- Computed prefill/decode token demand, safe per-worker throughput, and phase targets independently, then used the bottleneck as the aggregate target; continuously emitted production scale intents while preserving request shape, selected profile points, utilization, and blockers for decision audit.
- Decoupled Planner, ScaleIntent Controller, ScalingAdapter, IGD/ISI, and Scheduler; applied desired-vs-ready pending gates, one-step scaling, stable scale-down windows, cooldown, and stale-intent checks to prevent oscillation and concurrent writes.

Multi-Tenant Batch Inference Platform
OpenAI Batch API · Scheduling · Recovery

- Modeled batches as durable, recoverable asynchronous jobs rather than bulk request scripts: the database owns task truth while schedulers/workers claim, shard, execute, renew, and finalize work.
- Enabled multi-replica operation with Kubernetes Lease-based leader election, database-level claims, tenant/model concurrency budgets, and request-level error isolation, preventing duplicate execution and limiting offline impact on online serving.
- Implemented streaming JSONL processing, cancellation, expiry cleanup, output/error files, and failure-tolerant finalization from API contract through production monitoring.

Inference Router and Multi-Tenant Admission
P/D Routing · Priority Admission · Tracing

- Implemented P0-P3 admission in Rust with global/tier semaphores, strict priority scheduling, per-tenant round-robin, bounded queues, and wait timeouts; held permits through streaming response-body completion to avoid releasing capacity early.
- Integrated SGLang HTTP/gRPC P/D routing, selected compatible prefill/decode workers, and propagated bootstrap/transfer context; moved KV data directly through Mooncake/RoCE while preserving request-ID, trace-context, and transfer telemetry.

CORE TECHNOLOGIES

Inference Runtime & Routing	SGLang, vLLM, Mooncake Transfer Engine, RoCE/RDMA, KV event plane, KV-aware routing, P/D disaggregation, Dynamo/AIBrix architecture
Scheduling & Control Plane	Go, Kubernetes, CRDs/controllers, capability/capacity profiles, scale intents, reconciliation, readiness, finalizers, leader election, Helm, ArgoCD
Gateway & Async Serving	Python, Rust, OpenAI-compatible APIs, Batch API, LiteLLM, Higress, Redis, MySQL, multi-tenant concurrency, failure recovery
Observability & Performance	Prometheus, request tracing, QPS/TTFT/TPOT, ISL/OSL, KV hit rate, KV transfer, throughput curves, distributed-systems debugging

EDUCATION

Arizona State University — May 2019 - Dec 2020
Master of Computer Science, GPA 3.97 / 4.00

Arizona State University — Aug 2015 - May 2019
B.S. in Computer Science

INTELLECTUAL PROPERTY

Co-inventor on two granted invention patents, including an elastic cloud model-inference serving system; contributor to two authorized software copyrights.